COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Algorithms for maps, simulation and tools

Module 14 / expanded Volume 03

Four lessons, worked examples, full practical requirements and 20 lesson questions.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

14.1 — Searching, sorting and algorithm selection 3

14.2 — Graph traversal and dependency ordering 6

14.3 — Dijkstra, A* and admissible heuristics 9

14.4 — Dynamic programming, complexity limits and approximation 12

Continuity with the original program 15

Primary reference directory 15

Reading by topic 17

Module 14 — Algorithms for maps, simulation and tools

Prerequisites: Modules 06, 09 and 13.

Project: Build a pathfinding visualizer and asset-dependency planner. Provide correctness arguments, unreachable cases, complexity analysis and measured comparisons on multiple map families.

Primary references: R13, R06. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

14.1 — Searching, sorting and algorithm selection

Learning outcome: Choose an algorithm from its preconditions and the data distribution.

Choose from preconditions, not algorithm prestige

Binary search is useful when the range is ordered under the same comparison used by the search. Repeatedly halving an unsorted range has no correctness basis. Sorting first has a cost, so the best choice depends on whether there is one query or many, how often data changes and whether the sorted representation is useful elsewhere.

Linear search can be entirely appropriate for a short or frequently changing list. A game frame's actual workload may be dominated by memory access or maintaining an index rather than the comparison count. Start with a correct simple baseline and identify the measured bottleneck.

For sorting, compare not only asymptotic behavior but stability, auxiliary storage, input distribution and the cost of comparisons or moves. An educational insertion sort makes shifts visible and can exploit nearly sorted input; merge sort provides a predictable divide-and-combine structure. Standard algorithms remain the production baseline. [R13: searching, sorting]

Half-open intervals prevent many off-by-one errors

The reference lower-bound search maintains a candidate interval [lo,hi), with 0 <= lo <= hi <= n. The answer is the first position whose element is not less than the target, or n if all elements are less. Choose `mid = lo + (hi-lo)/2` to avoid adding two potentially large indices.

If `a[mid] < target`, the answer must lie after mid, so set lo = mid+1. Otherwise the answer lies at mid or earlier, so set hi = mid. Each iteration shrinks the interval. When lo == hi, that position is the lower bound.

This lower-bound result is an insertion position, not automatically a successful membership result. Membership additionally requires position < n and equality under the chosen key comparison. Duplicates are not an error: lower bound returns the first equivalent position.

Correct sorting means order and preservation

A sorted output can still be wrong if it drops an element or duplicates one. Verify that no later element compares less than its predecessor and that the output multiset equals the input multiset. A stable sort additionally preserves relative input order among equivalent keys.

Test stability using records with both a sort key and an original sequence ID. Sorting only raw integers cannot reveal whether equal-key records exchanged order. If a deterministic tie-breaker is part of the comparator, the elements may no longer be equivalent under that comparator, which changes what stability is testing.

For insertion sort, shift larger prior elements and place the held item into the gap. Using a strict comparison when deciding to shift can preserve equal-key order. For merge sort, choosing the left element on equal keys preserves stability across the merge when the recursive halves are stable.

Compare distributions with a reproducible experiment

Use the same inputs for both candidate sorts: empty, one element, duplicates, random values, already sorted, reverse sorted and nearly sorted. Save the random seed or actual input. Validate every output before accepting a timing.

Separate copying input, allocating scratch memory and sorting according to the declared experiment. A fair comparison can include all three as an end-to-end operation, or isolate sorting with prepared storage, but cannot charge setup to only one variant without saying so.

Report the input sizes and distribution, not just an algorithm name and one number. A sort that performs well on nearly sorted data may behave differently on reverse input. The module project asks for multiple map and data families precisely to prevent a favorable single case from becoming an unsupported general claim.

Lower bound of 4 in [1,4,4,9]

Begin lo = 0, hi = 4. Mid = 2, value 4 is not less than 4, so hi = 2. Mid = 1, value 4 is not less, so hi = 1. Mid = 0, value 1 is less, so lo = 1. Now lo = hi = 1, the first position of an equivalent value.

For target 10, the final position is 4, which is a valid insertion position but not an element index. For an empty vector, lo and hi start equal and the loop never reads an element.

For a sorting preservation test, input [2,1,2] must produce [1,2,2], not merely any nondecreasing vector. The output [1,2] is ordered but loses multiplicity.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_14_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
std::size_t lower_bound_index(std::span<const int> a, int x) {
    std::size_t lo = 0, hi = a.size();
    while (lo < hi) {
        const auto mid = lo + (hi - lo) / 2;
        if (a[mid] < x) lo = mid + 1;
        else hi = mid;
    }
```

```
    return lo;
}
```

Common failure modes

An insertion position equal to size is not safe to dereference. Sortedness does not prove element preservation. A stable-sort test needs distinguishable equal-key records. A benchmark without a recorded input distribution is difficult to interpret.

Practical assignment

Original requirement: Implement binary search with a documented interval convention and test empty, one-element, duplicate and missing cases. Compare two sorts on random and nearly sorted inputs. Pass: verify output order and element preservation.

1. Implement lower bound with a written half-open invariant and a separate membership wrapper.
2. Test empty, singleton, duplicate and missing targets against the standard lower-bound algorithm.
3. Implement or inspect insertion and merge sorts, checking order, multiset preservation and stability as applicable.
4. Compare random and nearly sorted inputs with saved data and repeatable Release measurements.

Required cases: empty -> insertion position 0; target below all -> 0; duplicate target -> first equivalent position; target above all -> size; sorting preserves duplicate counts; equal-key records preserve order for stable implementation.

Evidence to submit: Binary-search proof sketch, boundary tests, sorting property tests and distribution-specific comparison.

Independent transfer: The comparator sorts asset names case-insensitively, but search uses case-sensitive comparison. Explain why the search precondition is broken.

Assigned primary reading: R13: binary search, sorting and comparison models.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What precondition does ordinary binary search require?
2. What does stable sorting preserve?
3. Why test element preservation as well as sortedness?
4. What extra check turns a lower-bound position into a membership result?
5. Why test a sort on records containing original sequence IDs?

14.2 — Graph traversal and dependency ordering

Learning outcome: Use graph algorithms with explicit edge semantics.

An edge must have a declared meaning

A graph contains vertices and edges, but the engine supplies their semantics. In a navigation graph, an edge can mean a legal move. In an asset graph, edge A -> B can mean A must be built before B, or it can mean A depends on B. Both conventions are possible and produce opposite scheduling interpretations.

This bundle defines dependency edge prerequisite -> dependent. The topological order must place the edge's source before its destination. Write this convention on the diagram and in the test fixture. A correct graph algorithm applied to reversed semantics can produce a perfectly valid but operationally wrong build order.

Validate vertex IDs at the API boundary. An edge that names an out-of-range vertex is invalid input, not a reason to index outside an adjacency array. Decide how duplicate edges and isolated vertices are represented and ensure indegree counting follows the same representation. [R13: graph representations, BFS, DFS]

Breadth-first search builds distance layers

For an unweighted graph, BFS starts from the source at distance zero, marks it discovered and processes a FIFO queue. Each undiscovered neighbor is marked when enqueued, assigned distance one greater and given a predecessor. Marking at enqueue time avoids repeatedly enqueuing the same vertex through several incoming edges.

The queue processes vertices in nondecreasing distance layers. When a vertex is first discovered, the path uses the fewest edges under the unweighted model. Recover a path by following predecessors from the goal back to the source and reversing the result.

A missing path is a normal result. Source equal to goal yields the one-vertex path if the source is valid. Do not fabricate a predecessor for an unreachable vertex or loop forever while reconstructing it. For weighted costs, fewest edges need not mean least total cost; that is the next lesson.

Topological sorting detects dependency cycles

Kahn's algorithm computes indegrees, starts with all zero-indegree vertices and repeatedly removes one, appending it to the order. Removing a vertex conceptually removes its outgoing edges and reduces each dependent's indegree. Newly zero-indegree vertices become available.

If the output contains fewer than V vertices, some vertices remain in a directed cycle or blocked by a cycle, so no complete topological order exists. Report a cycle/dependency failure instead of silently returning a partial schedule as success. A DFS-based algorithm can additionally construct a cycle witness; that is a useful extension.

Several valid topological orders may exist. To make the teaching output deterministic, choose the smallest available vertex first, using a suitable ordered frontier. That tie-breaking can add a logarithmic factor compared with a plain queue. State whether your complexity claim includes that choice.

Prove the bound for the actual representation

With adjacency lists and a simple queue, a full BFS visits each vertex at most once and scans each outgoing edge at most once, yielding O(V+E) work under constant-time queue operations. An adjacency matrix may require scanning V possible neighbors per vertex and therefore O(V^2) work.

Likewise, basic Kahn sorting with a queue can be O(V+E). A min-heap frontier provides a lexicographically controlled choice but changes the bound to include heap operations. Determinism does not require a heap if insertion and adjacency order are already fully specified; choose deliberately.

The tests validate a topological order by building a position array and checking position[u] < position[v] for every edge u -> v. Comparing only against one expected order can reject a different valid order unless that exact tie-breaker is part of the contract.

Navigation and dependencies use the same structure differently

For edges 0 -> 1, 0 -> 2, 1 -> 3 and 2 -> 3, BFS from 0 reaches 3 in two edges. With adjacency order [1,2], the predecessor of 3 may be 1. Both 0,1,3 and 0,2,3 are shortest paths, but a deterministic rule chooses one.

For the same graph interpreted as prerequisites, a topological order can be 0,1,2,3 or 0,2,1,3. Add edge 3 -> 0 and no topological order exists. BFS still terminates because discovery marks prevent repeated expansion.

Keep the two correctness claims distinct: shortest unweighted path is not the same property as dependency order.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_14_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
queue.push(source);
seen[source] = true;
while (!queue.empty()) {
    const auto u = queue.front(); queue.pop();
    for (const auto v : adjacency[u]) {
        if (seen[v]) continue;
        seen[v] = true; parent[v] = u;
        queue.push(v);
    }
}
```

Common failure modes

Reversed dependency semantics can invalidate a build plan without violating the graph algorithm. Marking only when popped can create repeated queue entries. A partial topological order is not success. Exact tie-breaking can change the complexity bound.

Practical assignment

Original requirement: Implement BFS and topological sorting. Pass: recover a shortest unweighted path, report unreachable goals, reject a cyclic dependency graph and justify the O(V+E) traversal bound for adjacency lists.

1. Specify graph direction, vertex domain and duplicate-edge policy.
2. Implement BFS with predecessor reconstruction and explicit unreachable results.
3. Implement Kahn topological sorting and reject cyclic inputs.
4. Validate path edges and every dependency relation, then justify the bound for your chosen frontier and representation.

Required cases: source equals goal; isolated unreachable goal; diamond graph with two shortest paths; directed cycle; self-loop cycle; disconnected DAG includes all vertices; invalid vertex rejected.

Evidence to submit: Annotated graphs, predecessor traces, order validator, cycle tests and representation-specific complexity argument.

Independent transfer: A planner uses a priority queue to choose the alphabetically smallest ready asset. Update its complexity analysis instead of retaining an unqualified O(V+E) claim.

Assigned primary reading: R13: graph representations, BFS, DFS and topological sorting.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. When does BFS find a shortest path by edge count?
2. Can a directed graph with a cycle have a topological order?
3. Why mark visited nodes?
4. Why mark a BFS vertex discovered when enqueuing rather than repeatedly when dequeuing?
5. How can you test a topological order without demanding one particular valid sequence?

14.3 — Dijkstra, A* and admissible heuristics

Learning outcome: Build a correct pathfinder before optimizing its queue.

Shortest path requires a precise cost model

Dijkstra's algorithm applies to nonnegative edge costs. Maintain the best known distance g[v], initialize the source to zero and use a min-priority queue. When an edge u -> v with cost w offers a smaller g[u]+w, update g[v] and its predecessor and push a corresponding queue entry.

The reference uses a lazy queue: a vertex can have several entries, and a popped entry is skipped when its stored cost no longer equals the best known cost. This avoids needing a decrease-key operation in the introductory implementation. It also means queue size and operation counts differ from a one-entry-per-vertex design.

Validate nonnegative costs before searching. Check distance addition against the chosen numeric budget before computing it. An unreachable goal is not represented by adding to an integer maximum as though it were an ordinary distance. Use an explicit absence or a carefully controlled infinity sentinel. [R13: shortest paths]

A* adds a lower-bound estimate

A* prioritizes f(v) = g(v) + h(v), where h estimates remaining cost. An admissible heuristic never exceeds the true remaining optimal cost. A consistent heuristic satisfies h(u) <= w(u,v) + h(v) on each edge and normally has h(goal) = 0. Consistency supports a simple no-reopening closed-set policy; admissibility alone may require reopening improved nodes.

For a four-neighbor grid with minimum legal step cost c_min >= 0, Manhattan distance times c_min is a lower bound. With inexpensive diagonal moves, ordinary Manhattan distance can overestimate and must be reconsidered. With teleports or special movement rules, geometry alone may not bound route cost.

The teaching search permits improved nodes to be processed again and skips stale entries. With an admissible heuristic and the stated nonnegative-cost/goal conditions, stop when a valid goal entry is selected. For an untrusted arbitrary heuristic, returning the first goal is not a guarantee of optimality.

Construct a counterexample instead of merely warning

Consider S -> G of cost 10, S -> A of cost 1 and A -> G of cost 1. The optimal route costs 2 through A. Set h(A) = 100 and h(G) = 0. After expanding S, A has f = 101 while G has f = 10, so first-goal A* returns the cost-10 route. The heuristic deliberately overestimates and sacrifices optimality.

This is an experimentally checkable failure, not a vague claim that bad heuristics are risky. The lab includes it and compares with Dijkstra's cost 2. Then replace the heuristic with zero: A* becomes the same priority rule as Dijkstra and recovers the optimal result.

A pathfinder must also return a valid route: consecutive cells must be legal neighbors, no blocked cell may appear and the sum of actual edge costs must match the reported cost. Comparing only scalar costs can miss a corrupted predecessor chain.

Measure searches across map families

Test empty/open areas, corridors, obstacles creating detours, disconnected regions and weighted terrain. For small bounded graphs, an independent all-pairs or exhaustive reference can verify distances. Comparing A* only to Dijkstra is useful but can miss a defect shared by their common graph adapter.

Record nodes expanded, edges relaxed, queue pushes, peak queue size and elapsed time. A heuristic can reduce expansions but add per-node calculation cost. A map with one favorable corridor is not representative of all game navigation.

Tie-breaking affects which equal-cost path is returned and can affect expansion count without changing optimal cost. Define a deterministic order for the teaching fixture, then compare costs when several paths are valid. The capstone's correctness requirement is a valid minimum-cost path under the declared model, not a particular visual shape unless explicitly specified.

A weighted-grid baseline and an unsafe heuristic

On a four-neighbor unit-cost grid, from (0,0) to (3,4), Manhattan h = 7. Obstacles can only force an equal or longer route when they remove moves without adding cheaper shortcuts. On a weighted grid with minimum step cost 2, the corresponding lower bound is 14.

Now use the three-node graph S,A,G with costs 10,1,1 described above. Trace the frontier after S: (G,g=10,f=10), (A,g=1,f=101). First-goal A* with the bad heuristic returns 10; Dijkstra returns 2. Preserve this as a regression test showing the limitation of an assumption, not a failure to repair by hiding the test.

For every returned path, recompute its cost from the graph instead of trusting the stored scalar.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_14_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// Lazy-priority-queue relaxation, after checking cost bounds:
if (candidate < distance[v]) {
    distance[v] = candidate;
    parent[v] = u;
    frontier.push({candidate + heuristic(v), candidate, v});
}
// On pop: ignore entries whose stored g != distance[node].
```

Common failure modes

Negative edges violate ordinary Dijkstra assumptions. An overestimating heuristic can return a suboptimal first goal. A permanent closed set can be wrong for inconsistent heuristics without reopening. A correct cost alone does not prove the reconstructed route is valid.

Practical assignment

Original requirement: Implement Dijkstra and A* against the same grid model. Pass: compare path costs on many small maps, include no-path cases and demonstrate a deliberately overestimating heuristic that can sacrifice optimality.

1. Define neighbor rules, blocked cells, edge costs, distance bounds and tie-breaking.
2. Implement Dijkstra and A* with stale-entry handling and an explicit reopening policy.
3. Compare costs and independently validate routes over multiple small map families.
4. Demonstrate the overestimating-heuristic counterexample and record expansion/queue metrics without claiming universal speedup.

Required cases: start equals goal; unreachable region; weighted detour cheaper than direct route; zero-cost edge allowed; negative edge rejected; overestimating heuristic gives 10 versus optimal 2; A* with h=0 agrees with Dijkstra.

Evidence to submit: Cost-model specification, path validator, small-map oracle, counterexample trace and comparative metrics.

Independent transfer: Add diagonal movement with cost 1 while orthogonal movement also costs 1. Replace Manhattan with a valid lower bound and justify it.

Assigned primary reading: R13: shortest paths; R20: game navigation context.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What edge-cost condition does ordinary Dijkstra require?
2. What is Manhattan distance from (0,0) to (3,4)?
3. Why can inconsistent heuristics require reopening nodes?
4. For unit-cost diagonal and orthogonal moves on an unobstructed grid, why can Manhattan overestimate?
5. Why skip a priority-queue entry whose stored g differs from the current best distance?

14.4 — Dynamic programming, complexity limits and approximation

Learning outcome: Recognize repeated subproblems and justify a practical solution.

A state must contain enough history

Dynamic programming reuses solutions to overlapping subproblems. The state must retain everything needed for future decisions without carrying irrelevant history. For 0/1 knapsack, state (i,b) means the best value obtainable from the first i items with capacity b. Remembering only the remaining capacity while recursively choosing arbitrary items can accidentally allow reusing an item.

For item i with weight w_i and value v_i, compare excluding it with including it when it fits. The recurrence is best(i,b) = max(best(i-1,b), v_i + best(i-1,b-w_i)). The base row for no items has value zero. This recurrence explicitly permits each item at most once.

The proof is a case split on an optimal solution: it either excludes the last item or includes it, leaving a valid subproblem over earlier items. The recurrence is not justified merely because the resulting table looks plausible on one input. [R13: dynamic programming]

Evaluation order changes the problem if chosen wrongly

A full two-dimensional table stores every prefix/capacity state. A one-dimensional optimization can reuse the previous row, but for 0/1 knapsack it must update capacities in descending order. Ascending order can read a value updated using the current item and then use the same item again, solving a different unbounded-choice problem.

Use positive integer weights and nonnegative bounded integer values for the teaching implementation. Define a capacity budget before allocating a table and check value additions. Zero-weight and negative-value generalizations require an explicit contract rather than accidental behavior.

The included reference keeps the recurrence readable and compares with exhaustive search for small n. Reconstruction of chosen items is an extension that must verify each item appears at most once and that total weight and value match the reported optimum.

Pseudopolynomial is not polynomial in input bit length

The ordinary integer-capacity knapsack dynamic program uses O(nB) states or transitions up to constant factors, where B is the numeric capacity. If B is encoded in binary, its input length is proportional to log B. Therefore O(nB) is pseudopolynomial, not a general polynomial-time result in the bit length of the instance.

NP-completeness refers to a decision-problem family under formal reductions; it does not mean every individual instance is intractable. An optimization problem is usually described as NP-hard rather than casually treating its output as a yes/no language. Likewise, polynomial complexity does not guarantee that an algorithm fits a 16-millisecond frame budget.

Choose a bounded exact solver for small instances, a precomputed result when inputs change rarely, or an approximation/heuristic with a clear quality evaluation. Do not claim an approximation guarantee from a handful of good outputs. A formal guarantee needs a proof under stated assumptions. [R06; R13]

A simple oracle is a long-term engineering asset

Exhaustive search enumerates all subsets for tiny item counts and computes the best feasible value. Its exponential cost is acceptable when n is intentionally capped in tests. Because its structure differs from the dynamic program, it can reveal recurrence, loop-order and boundary defects.

Generate small deterministic cases, compare both values and validate reconstructed selections when present. Include an item too heavy to fit, a capacity of zero, equal-valued alternatives and a case where a greedy value-to-weight choice loses.

The broader lesson applies to graphics, simulation and tooling: preserve a slow clear baseline for small inputs. Optimize the production path while continuing to compare it against the baseline. Correctness oracles are not obsolete just because they are too slow for the largest runtime workload.

Loadout with capacity five

Items have (weight,value) pairs (2,3), (3,4) and (4,5). Exhaustive feasible choices include the empty set with value 0, each single item with values 3,4,5, and the first two together with weight 5 and value 7. The optimal value is 7.

In the dynamic program, after the first two items, state capacity five compares excluding item two (value 3) with including it plus the best capacity-two solution (4+3=7). Item three cannot improve that result: its value 5 plus the best remaining capacity one gives 5.

For one item of weight 2 and value 3 with capacity 4, descending one-dimensional updates produce 3. Ascending updates can incorrectly produce 6 by reusing the item. Keep this as a targeted regression case.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_14_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// For each positive-weight item in 0/1 knapsack:
for (std::size_t b = capacity; b >= weight; --b) {
    best[b] = std::max(best[b], value + best[b - weight]);
    if (b == weight) break; // avoid unsigned underflow
}
```

Common failure modes

Insufficient state changes the problem. Ascending capacity updates can turn 0/1 knapsack into repeated-item selection. O(nB) is pseudopolynomial in binary-encoded capacity. A heuristic's good example is not an approximation theorem.

Practical assignment

Original requirement: Solve a small 0/1 knapsack instance by exhaustive search and dynamic programming. Pass: compare answers, derive state count, and describe how a heuristic would be evaluated without claiming a guarantee it lacks.

1. Define item and capacity domains and derive the include/exclude recurrence.
2. Implement exhaustive search with a small explicit item-count cap.
3. Implement dynamic programming, checking capacity allocation and value arithmetic.
4. Differentially test small instances and compare one heuristic without overstating its guarantee.

Required cases: no items -> 0; capacity zero -> 0; all items too heavy -> 0; (2,3),(3,4),(4,5), B=5 -> 7; single (2,3), B=4 -> 3, not 6; many deterministic small cases agree with exhaustive solver.

Evidence to submit: State definition, recurrence proof, loop-order explanation, oracle comparisons and quality evaluation of a heuristic.

Independent transfer: The loadout now limits both mass and electrical power. Define the new sufficient state and explain the growth in table size before implementing it.

Assigned primary reading: R13: dynamic programming; R06: reductions and asymptotic reasoning.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What two features motivate dynamic programming?
2. Does NP-complete mean no instance can be solved quickly?
3. Why keep a brute-force solver for tiny inputs?
4. Why must one-dimensional 0/1 knapsack update capacity downward?
5. Why is O(nB) called pseudopolynomial when B is binary encoded?

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.